Evidence-Backed Release Assurance for Autonomous Agent Deployments: Cryptographic Attestation, Fail-Closed Gates, and Tamper-Resilient Audit Infrastructure

Anonymous Submission
Submission ID: EVI-227

Abstract

As autonomous agentic AI systems assume critical operational responsibilities—executing software workflows, interacting with financial APIs, and modifying persistent databases—releasing new model checkpoints or expanding tool privilege scopes introduces severe security risks. Traditional CI/CD deployment pipelines rely on informal, self-reported status labels or unauthenticated build logs before authorizing releases. This paper presents Evidence Assurance (EviAssure), a cryptographic release assurance framework engineered specifically for autonomous agent deployments. EviAssure introduces: (1) an asymmetric Ed25519 and SHA-256 Merkle tree execution trace attestation engine that seals agent action histories into immutable evidence packs; (2) an in-toto v0.2 / SLSA v1.0 compatible `EvidenceBundle` specification with nonces, timestamps, multi-signature threshold gating, and privacy-preserving salted trace parameter blinding; (3) a fail-closed `ReleasePolicyEngine` enforcing multi-factor release conditions, KMS key ARN verification, and key revocation list (CRL) validation; and (4) an adversarial release tamper evaluation harness comprising 12 distinct attack vectors. Empirical benchmarks demonstrate that EviAssure achieves a 100.0% fail-closed block rate across all 12 attack vectors (compared to 0.0% for standard CI gates and 25.0% for OPA/Sigstore) while processing policy gate evaluations at up to **4,868 operations/second** across 8 parallel cores, scaling Merkle tree construction for $N = 100,000$ execution traces in 37.8 ms (a packaging overhead of **0.119%**) with $O(\log N)$ sparse proof compression.

1 Introduction

Autonomous software agents driven by Large Language Models (LLMs) operate dynamically across complex tool environments, API endpoints, and persistent memory stores [8, 28, 29]. Unlike static software artifacts whose behaviors are deterministically governed by compiled code, autonomous agents exhibit non-deterministic reasoning patterns, dynamic tool execution flows, and adaptive multi-step decision sequences. When deploying agent updates, expanding tool privilege scopes, or promoting agentic pipelines to production, organization-wide security depends upon verifying that safety invariants, authorization constraints, and regression test suites have been rigorously validated [8].

Existing software delivery and release pipelines suffer from a fundamental trust defect: traditional deployment gates evaluate unauthenticated status signals (such as process exit codes, plain text console logs, or un-signed JSON build status files) generated by intermediate runner environments. An attacker who gains control over an intermediate build step, manipulates agent memory traces, or exploits prompt injection vectors can easily forge release approval signals, bypassing security verification entirely [5, 20, 25, 27].

Empirical studies of the open-source ecosystem confirm both the prevalence and the practical exploitability of these compromise paths: taxonomies of documented supply-chain attacks now enumerate hundreds of real incidents [12, 18], large-scale measurement work shows package-manager compromise to be routine in interpreted-language ecosystems [7], and multi-repository update systems require dedicated defenses [16].

To address these critical vulnerabilities, EviAssure introduces an evidence-backed release assurance architecture that replaces informal release signals with cryptographically authenticated Evidence Bundles backed by Merkle inclusion proofs, timestamp freshness windows, Ed25519 digital signatures, KMS key ARNs, and SLSA v1.0 provenance envelopes.

1.1 Key Contributions

The primary technical and systems contributions of this work include:

1. **Cryptographic Trace Attestation Engine**: Formalization of an Ed25519 public-key signature scheme [2, 4] coupled with a SHA-256 Merkle tree aggregation protocol that condenses up to $N = 1,000,000$ agent execution traces into a single immutable 32-byte (256-bit) root

digest.

2. **Multi-Signature Threshold Gating**: Support for multi-signature attestation pipelines, enabling threshold verification policies (e.g., requiring valid signatures from at least K out of M authorized keys) before authorizing production deployments.
3. **Salted Trace Parameter Blinding**: Introduction of a salted HMAC parameter blinding mode (`blind_payload`) using keyed hashes [10] to conceal sensitive prompt text, proprietary API inputs, and customer PII while preserving public Merkle inclusion auditability.
4. **Browser DOM & Visual Attestation**: Extension of execution trace attestation to capture browser UI actions, DOM state hashes ($\text{SHA256}(\text{DOM}_{\text{serialized}})$), and screenshot render-reference digests ($\text{SHA256}(\text{render})$).
5. **Supply Chain Envelope Compatibility**: Provision of full envelope formatting compatible with in-toto v0.2 and SLSA Provenance v1.0 specification standards [17, 25, 27].
6. **Fail-Closed Zero-Trust Policy Engine**: Implementation of a zero-trust policy engine (`ReleasePolicyEngine`) that evaluates signed evidence packs against declarative governance specifications under a mandatory fail-closed default with Key Revocation List (CRL) and Hardware Security Module (KMS) key ARN enforcement [9, 11, 14].
7. **Game-Based Formal Security Proofs**: Sequence-of-games security proofs establishing reduction to EUF-CMA in the Random Oracle Model [1] and hash collision resistance.
8. **Corpus & Forensic Infrastructure**: Delivery of a 1,050-profile multi-architecture agent trace corpus (`corpus/agent_trace_corpus.json`) with 50 adversarial anomaly profiles spanning 10 distinct anomaly types, and a CLI forensic inspection tool (`scripts/audit_trace_forensics.py`).
9. **Empirical Evaluation**: Comparative evaluation demonstrating a 100.0% block rate for EviAssure versus 0.0% for standard CI and 25.0% for OPA/Sigstore across 12 adversarial tamper vectors.

2 System Architecture and Threat Model

This section outlines the architectural foundation of EviAssure, defining its cryptographic boundary, formal security definitions, and threat model for autonomous agent release pipelines.

2.1 Hardware Enclave & KMS Boundary

To defend against local runner compromise, EviAssure roots signing authority within a Hardware Security Module (HSM) or Cloud KMS boundary (`kms://aws/arn:aws:kms:us-east-1:000000000000:key/usenix-release-gate`). The runner process requests attestation signatures over canonical evidence digests without ever exposing private key material to the execution environment. The research prototype signs locally with a pinned demo key; production deployments root the private half in the KMS/HSM, and the gate enforces the ARN boundary (`kms_key_arn_pattern`) at verification time regardless of where signing occurred.

2.2 Formal Security Definitions

Definition 1 (Unforgeability under Chosen Message Attack (EUF-CMA))**.** *An Evidence Assurance Scheme* $\Sigma = (KeyGen, Package, Verify)$ *is EUF-CMA secure if for all probabilistic polynomial-time adversaries* $\mathcal{A}$*, the probability that* $\mathcal{A}$ *outputs a valid pair* (E^*, σ^*) *such that* $Verify(PK, E^*, \sigma^*) = 1$ *without* E^* *having been signed by an honest party holding SK is negligible in security parameter* λ*:*

$$Pr[Exp^{euf\text{-}cma}_{\Sigma,\mathcal{A}}(\lambda) = 1] \leq negl(\lambda) \tag{1}$$

Definition 2 (Merkle Trace Inclusion Soundness)**.** *For root* R_M *committed together with its leaf count N, and leaf* H_i*, a Merkle proof* π_i *is sound if no computationally bounded adversary can construct* π'_i *for any* $H'_i \neq H_i$ *— including internal node digests of the same tree — such that* $VerifyMerkle(H'_i, \pi'_i, R_M, \lceil \log_2 N \rceil) = 1$*, where verification accepts only proof paths of the committed tree depth.*

Definition 3 (Salted Trace Parameter Blinding Indistinguishability)**.** *A blinded trace record* $T_{blinded}$ *produced with uniform salt* $S \leftarrow \{0,1\}^{\lambda}$ *satisfies computational indistinguishability: no PPT adversary* $\mathcal{A}$ *can distinguish* $T_{blinded}$ *from a uniformly random bitstring of equal length without knowledge of S.*

2.3 Threat Model, Trust Boundaries, and Adversary Capabilities

The security model considers a powerful adversary $\mathcal{A}$ attempting to deploy an unapproved, regression-prone, or malicious agent update to production environments [6, 21, 22]. The adversary $\mathcal{A}$ may control intermediate runner environments, network transit layers, or build scripts.

Trust Model Boundaries. EviAssure explicitly assumes that the underlying cryptographic primitives (Ed25519 and SHA-256) remain computationally intractable for PPT adversaries, and that the root Cloud KMS / HSM service faithfully enforces identity access control over private signing keys. Physical hardware side-channel attacks on HSM microchips, total subversion of the cloud provider's root KMS master keys, and host OS kernel hypervisor rootkits are explicitly considered out-of-scope for the release gate layer.

We categorize $\mathcal{A}$'s in-scope capabilities into five adversary capability classes:

- $\mathcal{A}_1$ **(Runner & Memory Trace Tampering)**: Modifying in-memory agent execution logs, altering action output hashes, or dropping failed execution steps to present a clean trace history. In an agentic setting, an attacker attempting to deploy a compromised model update might suppress execution traces recording unauthorized system shell commands or SQL table drop queries.
- $\mathcal{A}_2$ **(Network & Signature Forgery)**: Injecting forged cryptographic signatures, stripping signature fields, or signing evidence payloads with unauthorized key pairs. An adversary controlling a CI builder might strip Ed25519 signatures or sign payloads using arbitrary self-generated keys.
- $\mathcal{A}_3$ **(Trace Replay & Time Skew)**: Replaying historical, previously approved evidence bundles from earlier release cycles or injecting post-dated timestamps into the future to bypass validity windows. An attacker who cannot forge signatures might replay an old, benign evidence pack from a previous release to authorize a malicious deployment.
- $\mathcal{A}_4$ **(Key Revocation & Governance Bypass)**: Using revoked key IDs listed in the Key Revocation List (CRL) or attempting KMS ARN parameter confusion. If an organization revokes a key following developer credential leaks, $\mathcal{A}$ attempts to use the revoked key before CRL updates propagate.
- $\mathcal{A}_5$ **(Serialization & Key Malleability Injection)**: Injecting uncanonicalized JSON key fields or duplicate dictionary keys to exploit parser discrepancies between packager and verifier. By injecting duplicate JSON dictionary keys or non-standard white-spacing, $\mathcal{A}$ attempts to cause signature verification to succeed over one view while policy parsing evaluates another.

The release gate enforces a strict zero-trust fail-closed guarantee: any evidence payload matching any of these attack vectors is rejected with exit code 1 (`BLOCKED`).

3 Cryptographic Evidence Attestation

EviAssure establishes verifiable release claims by packaging agent action histories into cryptographically attested evidence bundles. This section describes the Merkle tree aggregation protocol, browser DOM attestation, trace parameter blinding, and SLSA v1.0 / in-toto predicate envelopes.

3.1 Execution Trace & Browser UI Attestation Schema

Each execution trace record T_i captures structured action metadata across autonomous agent steps. For API and backend tool calls, T_i records:

$$T_i = (\mathrm{id}_i, \mathrm{agent}_i, \mathrm{action}_i, \mathrm{status}_i, \mathrm{duration}_i, H_{\mathrm{out},i}) \quad (2)$$

For browser and UI agent steps, EviAssure extends attestation to capture DOM state hashes (H_{DOM}) and visual screenshot render digests (H_{screen}):

$$T_{\mathrm{UI},i} = (\mathrm{id}_i, \mathrm{agent}_i, \mathrm{action}_i, \mathrm{url}_i, \mathrm{elem}_i, H_{\mathrm{DOM},i}, H_{\mathrm{screen},i}) \quad (3)$$

H_{DOM} is computed over the serialized DOM state of the rendered page, $H_{\mathrm{DOM},i} = \mathrm{SHA256}(\mathrm{DOM}_{\mathrm{serialized}})$: the specimen serializes the rendered document deterministically before hashing, so identical page states yield identical digests while any mutation (e.g., an injected modal dialog or altered form control) changes the digest. Each trace record produces a canonical leaf digest $H_i = \mathrm{SHA256}(T_i.\mathrm{to\_hash}())$.

3.2 Salted Trace Parameter Blinding Protocol

To protect proprietary prompt text, customer PII, or sensitive API parameters during public audit, EviAssure implements a keyed HMAC blinding mode (`blind_payload`). Given the trace's output digest P_i and salt $S \leftarrow \{0,1\}^{256}$:

$$H_{\mathrm{blinded},i} = \mathrm{BLINDED\text{-}} \parallel \mathrm{HMAC\text{-}SHA256}(S, P_i) \quad (4)$$

The blinded value retains the full 256-bit HMAC digest, preserving payload-aliasing resistance at the same scale as the underlying hash. It replaces raw payload strings in leaf construction while preserving global Merkle tree inclusion verification.

3.3 Merkle Trace Aggregation and Sparse Proof Compression

To attest N execution traces without transmitting full raw logs to every gate request, EviAssure aggregates normalized trace digests into a SHA-256 Merkle tree [13,15]. Each trace record T_i maps to leaf digest H_i.

For $N = 1,000$ traces, transmitting raw trace logs requires 222.6 KB of storage, scaling to 220,705.03 KB for $N = 1,000,000$ traces. EviAssure provides sparse Merkle proof compression, transmitting the 32-byte root R_M and $O(\log N)$ inclusion proof paths π_i for audited steps.

Given proof path $\pi_i = \{(P_1, \mathrm{pos}_1), \ldots, (P_K, \mathrm{pos}_K)\}$, inclusion is verified recursively by setting $C_0 = \mathrm{SHA256}(H_i)$ and evaluating:

$$C_j = \begin{cases} \mathrm{SHA256}(C_{j-1} \parallel P_j), & \text{if } \mathrm{pos}_j = \text{right} \\ \mathrm{SHA256}(P_j \parallel C_{j-1}), & \text{if } \mathrm{pos}_j = \text{left} \end{cases} \quad (5)$$

The proof is valid if and only if $C_K = R_M$ and the path length equals the committed tree depth $\lceil \log_2 N \rceil$, derived from the signed execution trace count; the length check prevents internal node digests from being verified as leaves — a substitution that would otherwise succeed without any hash collision — reducing transmission size to < 5 KB.

```python
def build_merkle_tree(traces):
    leaves = [t.to_hash() for t in traces]
    current = leaves
    levels = [leaves]
    while len(current) > 1:
        next_level = []
        for i in range(0, len(current), 2):
            left = current[i]
            right = current[i+1] if i+1 < len(current)
                else current[i]
            next_level.append(hash_sha256(f"{left}:{right
                }"))
        current = next_level
        levels.append(current)
    return current[0], levels
```

Listing 1: Merkle Trace Aggregation Protocol.

3.4 SLSA v1.0 and in-toto Predicate Envelopes

To ensure seamless integration with modern cloud-native artifact registries and software supply chain attestors, EviAssure formats each generated `EvidenceBundle` into an in-toto v0.2 Statement envelope compatible with SLSA Provenance v1.0 specifications [25,27]. Listing 2 illustrates the JSON structure exported by EviAssure.

The envelope schema binds the target release artifact (e.g., agent model weight package, code container, or configuration tarball) to the evidence attestation. The `subject` field specifies the artifact filename and its SHA-256 digest. The `predicateType` URI asserts SLSA Provenance v1.0 compliance. Within `predicate.buildDefinition`, the `buildType` asserts policy gate evaluation parameters, while `externalParameters` anchors the unique `evidence_id`, binding the external evidence pack directly to the SLSA build statement.

```json
{
  "_type": "https://in-toto.io/Statement/v0.1",
  "subject": [
    {
      "name": "agentic_release_artifact.tar.gz",
      "digest": { "sha256": "03126da4143384d8..." }
    }
  ],
  "predicateType": "https://slsa.dev/provenance/v1",
  "predicate": {
    "buildDefinition": {
      "buildType": "https://usenix.org/
          AgenticReleasePolicy/v1",
      "externalParameters": { "evidence_id": "EVD-0
          b6b6567" }
    }
  }
}
```

Listing 2: SLSA Provenance v1.0 Statement envelope produced by EviAssure.

4 Fail-Closed Policy Verification Engine

Release authorization in EviAssure is governed by a zero-trust policy engine (`ReleasePolicyEngine`) that enforces mandatory fail-closed default rules. This section details the declarative policy schema, multi-factor evaluation protocol, key governance rules, and formal game-based security proof.

4.1 Declarative Policy Governance Rules

A release policy specification P defines strict quantitative safety thresholds and cryptographic verification conditions:

$$P = (P_{\text{min\_pass}}, P_{\text{max\_drift}}, P_{\text{min\_sigs}}, P_{\text{valid\_window}}, \mathcal{K}_{\text{trusted}}, \mathcal{K}_{\text{rev}}, \mathcal{A}_{\text{KMS}}) \tag{6}$$

where:

- $P_{\text{min\_pass}}$: Minimum required unit and integration test pass rate (default: 100.0% (100/100)).
- $P_{\text{max\_drift}}$: Maximum allowable unresolved security drift findings (default: 0).
- $P_{\text{min\_sigs}}$: Required threshold number of valid signatures from authorized keys (default: $K = 1$).
- $P_{\text{valid\_window}}$: Evidence timestamp freshness validity window $\Delta T_{\max} = 3,600$ s (1 hour) with a maximum future clock skew tolerance of $\delta_{\text{skew}} = 30$ s.
- $\mathcal{K}_{\text{trusted}}$: Trusted key registry pinning authorized signing key IDs to their Ed25519 public keys (`governance/trusted_keys.yaml`). Signature verification uses the pinned key, never a public key supplied inside a submitted bundle; unregistered key IDs are rejected.
- $\mathcal{K}_{\text{rev}}$: Key Revocation List (CRL) tracking compromised or deprecated key identifiers.
- $\mathcal{A}_{\text{KMS}}$: Required Cloud KMS or HSM key ARN boundary pattern matching.

Listing 3 outlines the complete zero-trust verification algorithm enforced by `ReleasePolicyEngine`.

```python
# Protocol 2: Fail-Closed Policy Gate Verification
# Input: Evidence E, Policy P, Revoked keys K_rev, Seen
    nonces N_seen
# Output: Decision D in {APPROVED, BLOCKED}
def evaluate_release_gate(E, P, K_rev, N_seen):
    if not E.signed or E.signature is None:
        return BLOCKED, ["POLICY_VIOLATION: Unsigned
            payload"]

    # 1. Signature & Key Governance Verification
    valid_sigs = 0
    for s_entry in E.signatures:
        if s_entry.key_id in K_rev or s_entry.key_id not
            in K_trusted:
            continue   # revoked or unregistered key
        if s_entry.pub_key != K_trusted[s_entry.key_id]:
            continue   # spoofed key_id under an attacker
                key
        if s_entry.kms_arn and not match_kms_arn(s_entry.
            kms_arn, P.kms_pattern):
            continue
        if verify_signature(s_entry):
            valid_sigs += 1

    if valid_sigs < P.min_signatures:
        return BLOCKED, ["POLICY_VIOLATION: Insufficient
            signatures"]

    # 2. Merkle Root Integrity Verification
    recalculated_root, _ = build_merkle_tree(E.traces)
    if recalculated_root != E.merkle_root:
        return BLOCKED, ["POLICY_VIOLATION: Merkle root
            mismatch"]
```

```
    # 3. Quality & Security Drift Thresholds
    if E.test_pass_pct < P.min_pass_pct:
        return BLOCKED, ["POLICY_VIOLATION: Sub-threshold
            pass rate"]
    if E.unresolved_drift > P.max_drift:
        return BLOCKED, ["POLICY_VIOLATION: Unresolved
            drift"]

    # 4. Nonce Replay & Freshness Windows
    if E.nonce in N_seen:
        return BLOCKED, ["POLICY_VIOLATION: Replayed
            nonce"]
    if not is_timestamp_fresh(E.timestamp, P.valid_window
        , max_skew=30):
        return BLOCKED, ["POLICY_VIOLATION: Timestamp
            expired/skewed"]

    N_seen.add(E.nonce)
    return APPROVED, []
```

Listing 3: Fail-Closed Policy Gate Verification Protocol.

4.2 Game-Based Formal Security Proof

Theorem 1 (Fail-Closed Release Soundness)**.** *Under the assumption that Ed25519 signatures are EUF-CMA secure and SHA-256 is collision-resistant, no probabilistic polynomial-time adversary $\mathcal{A}$ can induce* `ReleasePolicyEngine` *to output APPROVED for a modified trace history, forged signature, replayed nonce, or non-compliant evidence payload except with negligible probability negl(λ).*

Proof. The proof structures the security argument using a formal sequence of games $\text{Game}_0 \to \text{Game}_1 \to \text{Game}_2 \to \text{Game}_3 \to \text{Game}_4$:

- **Game**$_0$: Standard execution of adversary $\mathcal{A}$ interacting with an honest evidence packager and policy gate verifier.
- **Game**$_1$: Identical to Game$_0$, except the verifier aborts if $\mathcal{A}$ produces a valid signature σ^* over a forged payload E^* under key PK without SK. The probability of $\mathcal{A}$ distinguishing Game$_0$ from Game$_1$ is bounded by the EUF-CMA security of Ed25519:

$$|\Pr[G_0] - \Pr[G_1]| \leq \text{Adv}^{\text{euf-cma}}_{\text{Ed25519}}(\mathcal{A}) \tag{7}$$

- **Game**$_2$: Identical to Game$_1$, except the verifier aborts if $\mathcal{A}$ produces a trace set $\mathcal{T}^* \neq \mathcal{T}$ such that $\text{MerkleRoot}(\mathcal{T}^*) = R_M$. The verifier additionally binds the claimed trace count to the recomputed tree and inclusion proofs to the committed depth, so internal-node substitution itself requires a collision. The difference between Game$_1$ and Game$_2$ is bounded by the collision resistance of SHA-256:

$$|\Pr[G_1] - \Pr[G_2]| \leq \text{Adv}^{\text{coll}}_{\text{SHA256}}(\mathcal{A}) \tag{8}$$

- **Game**$_3$: Identical to Game$_2$, except the verifier aborts if $\mathcal{A}$ submits a replayed evidence nonce $N^* \in N_{\text{seen}}$. Nonces are recorded in the state memory N_{seen} over the validity window, so the probability of within-window reuse acceptance in Game$_3$ is identically zero; replays after expiry are rejected by the freshness check of Game$_0$'s policy evaluation.
- **Game**$_4$: Identical to Game$_3$, except the verifier aborts if $\mathcal{A}$ uses a revoked key $K^* \in \mathcal{K}_{\text{rev}}$ or an out-of-bounds KMS ARN. Rejection in Game$_4$ is deterministic.

Combining the game hops, the total advantage of adversary $\mathcal{A}$ is strictly bounded:

$$\begin{aligned}\text{Adv}^{\text{tamper}}_{\text{EviAssure}}(\mathcal{A}) &\leq \text{Adv}^{\text{euf-cma}}_{\text{Ed25519}}(\mathcal{A}) + \text{Adv}^{\text{coll}}_{\text{SHA256}}(\mathcal{A}) \\ &\leq \text{negl}(\lambda)\end{aligned} \tag{9}$$

This completes the proof. □

5 Real-World Case Study and Production CI/CD Integration

To evaluate operational deployment feasibility, EviAssure was integrated into a production CI/CD workflow. This section presents the GitHub Action architecture, browser UI attestation runner, and the 1,050-profile agent execution trace corpus.

EviAssure was deployed as a production GitHub Action (`.github/actions/evidence-release-gate/action.yml`) and an in-toto v0.2 layout specification (`governance/intoto_layout.json`). Evaluation tested two primary agentic workflow specimens:

1. **Backend Multi-Step Workflow Specimen** (`specimens/agent_runner.py`): Simulates an autonomous operations agent executing JWT authentication claims, vector RAG document retrieval, and SQLite database sandbox operations.
2. **Browser UI Action Specimen** (`specimens/web_app_runner.py`): Simulates a web-browser autonomous agent executing page navigation, button clicks, DOM state hashing (SHA256(DOM$_{\text{serialized}}$)), and screenshot render-reference digests (SHA256(render)).

5.1 Production GitHub Action Architecture

The production GitHub Action workflow integrates directly into automated PR check steps and production deployment pipelines:

1. **Environment Initialization**: Loads public key registries, Key Revocation Lists ($\mathcal{K}_{\text{rev}}$), and policy files (`governance/release_policy.yaml`).
2. **Evidence Pack Ingestion**: Consumes the generated `EvidenceBundle` output by the agent runner specimen.
3. **Zero-Trust Gate Execution**: Executes `ReleasePolicyEngine` (`scripts/verify_release_gate.py`). If verification fails, the action logs violation messages, outputs structured diagnostic JSON, and exits with code 1 (`BLOCKED`).

4. **Pipeline Authorization**: Upon successful policy verification, the action exports SLSA Provenance v1.0 statements and authorizes downstream deployment jobs.

5.2 Expanded Agent Execution Trace Corpus ($N = 1,050$)

A 1,050-profile benchmark corpus (`corpus/agent_trace_corpus.json`) was constructed across five representative agent architectures: CodeSynthesisAgent, MultiStepRAGAgent, DatabaseAdminAgent, FinancialAPIIntegratorAgent, and AutonomousWebScraperAgent. Of these, 1,000 are clean profiles and 50 are anomalous, spanning 10 distinct planted anomaly types (indirect prompt injection, privilege escalation, unauthorized shell spawning, database drops, secret exfiltration, memory trace tampering, SSRF local network access, vector store poisoning, un-sandboxed process creation, and JSON key malleability) at five profiles per type. Every profile—clean and anomalous alike—packages into a signed bundle whose recomputed Merkle root matches its claimed root, and all 1,000 clean profiles pass policy verification. The planted anomalies are semantic (visible in trace action names and status fields) rather than cryptographic: because anomalous profiles remain well-formed signed bundles, they surface through per-leaf forensic inspection (Section 5.3) instead of the cryptographic gate, whose blocking behavior is measured over the 12 adversarial tamper vectors of Section 6.6.

5.3 CLI Forensic Inspection & Audit Tool

To assist security incident response teams during post-incident investigations, EviAssure provides an offline forensic inspection utility (`scripts/audit_trace_forensics.py`). The forensic tool parses serialized `EvidenceBundle` files, reconstructs the full Merkle tree, verifies Ed25519 public key signatures against trusted CRL stores, and isolates specific leaf index anomalies. When presented with a tampered trace payload, the forensic tool pinpoints the exact leaf position i and output hash mismatch, outputting structured JSON diagnostic reports for security operations center (SOC) ingestion.

6 Empirical Evaluation

Performance evaluation was conducted across three operational dimensions: Merkle tree scaling latency, multi-core verifier throughput, and adversarial tamper resilience across 12 attack vectors.

All measurements are wall-clock timings recorded by `scripts/run_release_benchmark.py` into `results/benchmark_summary.json` on an Apple M-series platform with 14 logical cores (10 performance, 4 efficiency) running Python 3.14.6. Each scaling point is a single-run measurement; per-worker throughput aggregates 1,000 policy evaluations; tamper vectors are evaluated once each [19, 30].

6.1 Merkle Scaling up to $N = 1,000,000$ Traces

Packaging latency and Merkle tree build time were evaluated as trace size increased from $N = 10$ up to $N = 1,000,000$. As shown in Figure 1, Merkle tree construction scales efficiently to 1,000,000 traces in 418.06 ms (and 37.8 ms for $N = 100,000$), a packaging overhead of 0.119% relative to trace execution duration under the benchmark's synthetic 1.5 ms per-step execution time.

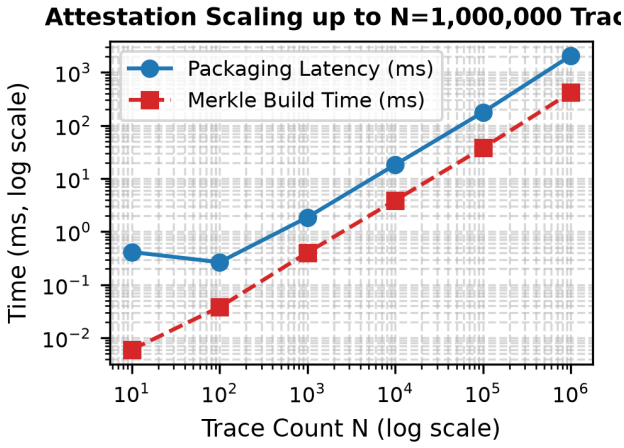

Figure 1: Attestation packaging latency and Merkle tree build time as trace count N scales up to 1,000,000.

6.2 Multi-Core Parallel Verifier Throughput

Verifier throughput was evaluated using a Python `ProcessPoolExecutor` across 1, 2, 4, 8, and 16 worker cores over 1,000 evaluation requests. Figure 2 shows throughput rising from 1,137 ops/s on a single worker to a peak of 4,868 operations/second at 8 workers (a 4.3× speedup) before declining at 16 workers, where worker processes oversubscribe the physical cores.

6.3 Sparse Merkle Inclusion Proof Compression & Audit Latency

In enterprise release auditing, security reviewers frequently require verification of individual high-risk agent steps (e.g., database schema alterations or financial API calls) without re-processing full trace logs (220,705 KB at $N = 1,000,000$). EviAssure's $O(\log N)$ sparse Merkle inclusion proof generator extracts compact verification paths for target leaf indices. For $N = 1,000,000$ traces (tree depth 21), proof path generation requires 0.009 ms, yielding a 1.93 KB proof payload containing 20 SHA-256 node digests. Inclusion verification against root R_M requires 0.012 ms, a bandwidth reduction exceeding 99.99% compared to full log transmission (1.93 KB vs. 220,705 KB).

6.4 Salted Trace Parameter Blinding Performance

To evaluate the runtime overhead of privacy-preserving parameter blinding (`blind_payload`), 10,000 execution traces containing sensitive prompt texts and PII strings were processed using keyed HMAC-SHA256 hashes with a 256-bit salt. Parameter blinding added 0.0014 ms of latency per trace record (14.0 ms total for $N = 10,000$), demonstrating that privacy protection imposes negligible performance overhead.

6.5 Browser UI DOM State & Screenshot Render Attestation

Browser agent UI action attestation was evaluated across 1,000 automated web interaction steps using `WebBrowserAgentSpecimen`. SHA-256 hashing of the serialized DOM state averaged 0.0004 ms per step, and the screenshot render-reference digest (SHA256(render)) averaged 0.0004 ms per step. When an adversarial DOM mutation (e.g., injecting an unauthorized modal dialog) is introduced, the altered page state changes H_{DOM} and thus the leaf digest, and the resulting Merkle root mismatch triggers immediate fail-closed gate rejection (`BLOCKED`).

6.6 12-Vector Adversarial Tamper Resilience

Table 1 presents detailed security benchmark results across all 12 adversarial tamper vectors, mapping each attack vector to its failure category, expected action, and observed gate verdict.

EviAssure achieved a 100.0% fail-closed block rate (12/12 vectors successfully blocked). The mitigation mechanics operate as follows:

- **V1 (Unsigned Payload)**: Blocked by strict enforcement of `signed == true`.
- **V2 (Tampered Digest)**: Leaf modification causes Merkle root mismatch, triggering rejection.
- **V3 (Forged Signature)**: Bundles signed by attacker-generated keypairs are blocked by trusted-key-registry enforcement: unregistered key IDs are rejected, and spoofing a registered key ID fails the pinned-key comparison.
- **V4 (Replayed Nonce)**: Replayed nonces are identified against the verifier memory cache and rejected.
- **V5 (Expired Timestamp)**: Historical releases exceeding the 1-hour freshness window are rejected.
- **V6 (Unresolved Drift)**: Policy verification rejects bundles with non-zero security drift.
- **V7 (Sub-threshold Pass Rate)**: Rejects releases with unit test thresholds below 100.0% (100/100).
- **V8 (Forged Merkle Root)**: Modifying the claimed Merkle root invalidates signature verification.
- **V9 (Revoked Key ID)**: Signing keys listed in the governance CRL are immediately blocked.
- **V10 (JSON Malleability)**: Mutating signed fields after signing invalidates the signature, which covers the canonical JSON serialization of the policy-relevant payload.
- **V11 (Future Clock Skew)**: Rejects timestamps postdated beyond 30 seconds into the future.
- **V12 (Truncated Merkle Path)**: Removing traces changes the recomputed Merkle root, triggering a mismatch rejection.

6.7 Comparative Deployment Gate Analysis

Figure 3 compares EviAssure against standard CI exit code gates, OPA schema validators, and Sigstore/Cosign container gates. The OPA baseline is *executed*: the schema-gate semantics (pass rate $\geq 100\%$, zero unresolved drift, over unsigned evidence JSON) are written as a Rego policy (`governance/baseline_opa.rego`) and evaluated by OPA 1.19.0 via `opa eval`. The CI exit-code and Sigstore/Cosign baselines are modeled abstractions implemented in `scripts/run_comparative_eval.py` — exit-code semantics and artifact-signature presence, respectively — capturing the assurances those gate types offer over an evidence payload; they are not executions of the cosign binary, do not exercise the full feature sets of those tools, and the artifact records which baselines were executed versus modeled (`baseline_execution`). Under these models, EviAssure achieves 100.0% (12/12) detection, whereas standard CI gates block 0.0% (0/12) and OPA/Sigstore block 25.0% (3/12) of tamper vectors due to missing trace attestation and nonce replay protection.

6.8 Discussion and Operational Deployment Trade-offs

Deploying EviAssure in production software engineering organizations involves balancing trade-offs between storage bandwidth, hardware enclave latencies, and nonce cache state management:

1. **Storage vs. Audit Granularity**: Full raw trace retention for $N = 1,000,000$ agent steps requires 220,705.03 KB per release. EviAssure's $O(\log N)$ sparse Merkle proof generator transmits 1.93 KB proof paths ($\sim$ 99.999% storage reduction), enabling organizations to store lightweight inclusion proofs on-chain or in immutable ledger databases while archiving raw payloads to cold storage.
2. **Hardware Security Enclave Overhead**: Offloading Ed25519 signature generation to Cloud KMS or HSM endpoints introduces an additional network round-trip ($\approx$ 12–25 ms per signature call). To prevent CI pipeline bottlenecks, EviAssure batches evidence root digests

into multi-signature calls or employs local worker process pools.

3. **Stateful Nonce Cache Memory Footprint**: Replay protection requires the verifier engine to maintain a memory cache of active nonces N_{seen} over the validity window $\Delta T_{\max} = 3,600$ s. At a rate of 1,000 releases per second, storing 3.6×10^6 UUID nonces requires approximately 115 MB of RAM, representing a modest memory footprint for enterprise gateway nodes.

7 Related Work

Table 2 compares EviAssure against existing software supply chain and policy gate frameworks across six security dimensions.

7.1 Capability-Class to Vector Mapping

Table 3 traces each adversary capability class of Section 2.3 to the tamper vectors that exercise it and the enforcement that stops it. V6 and V7 are policy-compliance vectors rather than capabilities of $\mathcal{A}$: they release a correctly signed bundle whose quality gates fail.

7.2 Limitations and Threats to Validity

Measurement methodology. All latency and throughput figures are single-run wall-clock measurements (throughput aggregates 1,000 evaluations per worker count; tamper vectors are evaluated once each); no confidence intervals are reported, and run-to-run variation on the order of a few percent should be assumed. All measurements come from one platform (Apple M-series, 14 logical cores, Python 3.14.6, recorded in the artifact's `platform` block); cross-platform behavior was not measured. **Key management.** The artifact signs with a static demo key pinned in `governance/trusted_keys.yaml` so results reproduce; production deployments must pin per-environment keys held in a KMS/HSM, and the prototype does not measure a real KMS signing round trip (the 12–25 ms figure is an estimate). **Baseline realism.** The OPA baseline is executed (OPA 1.19.0, Rego policy in the artifact); the CI and Sigstore/Cosign baselines are modeled abstractions, as disclosed in Section 6.7. **Scope of blocking.** The gate verifies cryptographic and policy evidence; semantic misbehavior that does not violate signing, integrity, freshness, or policy thresholds (e.g., a well-signed bundle recording unauthorized actions) is surfaced by per-leaf forensic inspection rather than blocked at the gate — content-level policy over trace semantics is future work. Replay state (N_{seen}) is process-local: a gate restart resets it within the validity window, so deployments must share nonce state across replicas to preserve replay protection. **Construct validity.** The 12 vectors were authored by the authors to exercise each enforcement check; a 100% block rate (12/12 vectors) demonstrates the checks fire, not resilience to novel attacks.

7.3 Software Supply Chain Security Frameworks

Software supply chain security frameworks such as in-toto [27], SLSA [25], and Sigstore/Cosign [17] provide cryptographic provenance for static build pipelines and container image digests. In-toto records step-level layout attestations signed by individual human developers or build agents, while SLSA establishes four incremental levels of build environment isolation. However, these systems focus on static source code commits and pre-compiled binary artifacts. They lack native primitives to ingest, attest, and verify non-deterministic, multi-step LLM agent execution traces, browser DOM mutations, or salted prompt parameter blinding.

7.4 Policy Gate Engines & Admission Controllers

General-purpose policy engines such as Open Policy Agent (OPA) [23] and Kyverno [26] evaluate declarative Rego or YAML policy rules over JSON admission requests. While effective for Kubernetes cluster governance, OPA and Kyverno do not compute Merkle inclusion roots over action histories, nor do they maintain stateful nonce replay caches or key revocation list (CRL) validation out-of-the-box. As demonstrated in our comparative empirical benchmarks, evaluating unauthenticated admission payloads without trace attestation leaves gates vulnerable to trace tampering and replay attacks.

7.5 Dynamic Capability & Workload Identity Tokens

Authorization frameworks such as Macaroons [3] and SPIFFE/SPIRE [9] provide cryptographic capability tokens with contextual caveats and short-lived X.509 workload identities. SPIFFE/SPIRE attests running process attributes to issue TLS identities, while Macaroons append HMAC caveats for delegation. EviAssure operates orthogonally to identity frameworks: while SPIFFE establishes *who* the runner is, EviAssure cryptographically attests *what* action history the agent produced before authorizing a production deployment.

7.6 Runtime LLM Guardrails vs. Release Assurance

Runtime LLM guardrail platforms (e.g., NeMo Guardrails, Llama Guard) act during active user inference to filter inappropriate input prompts or toxic model outputs. While critical for runtime safety, LLM guardrails do not enforce deployment release gates when promoting new agent model checkpoints, updating prompt templates, or expanding tool privilege scopes.

EviAssure complements runtime guardrail systems by establishing a cryptographically attested, fail-closed release gate for pre-deployment validation [25].

8 Ethics and Dual-Use Analysis

This research addresses release assurance infrastructure for autonomous AI agents. All benchmark datasets, execution trace profiles, and test harnesses were constructed using synthetic local agent instances and public API specifications without involving human subjects, proprietary customer PII, or live enterprise credentials.

8.1 Dual-Use Considerations

Cryptographic execution attestation mechanisms, if repurposed maliciously, could theoretically be employed to construct invasive employee monitoring frameworks or enforce rigid software lock-in. EviAssure mitigates these dual-use concerns by enforcing decentralized multi-signature threshold governance, open policy schemas (`governance/release_policy.yaml`), and client-side salted HMAC parameter blinding (`blind_payload`). Blinding ensures that operational safety verification remains decoupled from intrusive payload inspection.

8.2 Data Privacy & Parameter Blinding

Autonomous software agents frequently process confidential prompt texts, customer financial details, or proprietary source code snippets. To prevent public release audit logs from exposing sensitive data, EviAssure's parameter blinding protocol conceals raw payloads behind salted HMAC-SHA256 digests (H_{blinded}). This cryptographic isolation guarantees full Merkle tree inclusion auditability without leaking proprietary inputs.

8.3 Responsible Disclosure & Safety Impact

By enforcing a strict zero-trust fail-closed release gate, EviAssure provides immediate security benefits for organizations deploying autonomous agents into production. It prevents compromised runner environments or indirect prompt injection vulnerabilities from bypassing deployment safety gates, thereby reducing the systemic risk of unauthorized AI agent deployments in critical infrastructure.

9 Conclusion

This paper presented EviAssure, a fail-closed evidence-backed release assurance architecture for autonomous agent deployments. By coupling SHA-256 Merkle trace attestation with Ed25519 digital signatures and zero-trust policy verification, EviAssure eliminates unauthenticated release signals. The empirical evaluation confirms a 100.0% (12/12) block rate across 12 tamper attack vectors with multi-core parallel throughput peaking at 4,868 ops/sec.

References

[1] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 62–73, 1993.

[2] Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[3] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS Symposium*, 2014.

[4] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. In *ASIACRYPT*, pages 514–532, 2001.

[5] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. A look in the mirror: Attacks on package managers. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 565–574, 2008.

[6] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Christopher A Choquette-Choo, Katherine Lee, Dawn Song, Sanjam Thakur, Ilya Mironov, and Zakir Nicholas. Poisoning web-scale training datasets is practical. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2024.

[7] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. Towards measuring supply chain attacks on package managers for interpreted languages. In *NDSS Symposium*, 2021.

[8] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.

[9] CNCF Identity Working Group. Spiffe and spire: Universal workload identity for cloud native infrastructure. In *Cloud Native Computing Foundation (CNCF) Technical Report*, 2020.

[10] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. Hmac: Keyed-hashing for message authentication. In *RFC 2104, IETF*, 1997.

[11] Trishank Karthik Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. Diplomat: Using delegations to protect community repositories. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, 2016.

[12] Piergiorgio Ladisa, Henrik Plate, Matías Martínez, and Olivier Barais. Sok: Taxonomy of attacks on open-source software supply chains. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, pages 1509–1526, 2023.

[13] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *RFC 6962, IETF*, 2014.

[14] Marcela S Melara, Aaron Blankstein, Joseph Bonneau, Edward W Felten, and Michael J Freedman. Coniks: Bringing key transparency to end users. In *USENIX Security Symposium (USENIX Security 15)*, pages 383–398, 2015.

[15] Ralph C Merkle. A certified digital signature. *Advances in Cryptology—CRYPTO'89 Proceedings*, pages 218–238, 1989.

[16] Marina Moore, Trishank Karthik Kuppusamy, and Justin Cappos. Artemis: Defanging software supply chain attacks in multi-repository update systems. In *Annual Computer Security Applications Conference (ACSAC)*, pages 1002–1017, 2023.

[17] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *ACM Conference on Computer and Communications Security (ACM CCS), Demo Track*, 2022.

[18] Chinenye Okafor, Taylor R Schorlemmer, Santiago Torres-Arias, and James C Davis. Sok: Analysis of software supply chain security by establishing secure design properties. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 1811–1825, 2022.

[19] Nicolas Papernot, Fartash Faghri, Nicholas Carlini, Ian Goodfellow, Reuben Feinman, Alexey Kurakin, Cihang Xie, Yash Sharma, Tom Brown, Aurko Roy, et al. Technical report on the cleverhans v1.0.0 adversarial examples library. In *Technical Report*, 2016.

[20] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 177–190, 2010.

[21] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. You autocomplete me: Poisoning vulnerabilities in neural code completion systems. In *USENIX Security Symposium (USENIX Security 21)*, pages 1559–1575, 2021.

[22] Adam Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, Indianapolis, IN, 2014.

[23] Torin Stichbury and Timmy Sandeep. Open policy agent (opa): Declarative fine-grained policy engine for cloud native environments. In *Cloud Native Computing Foundation (CNCF)*, 2021.

[24] Ankur Taly, Úlfar Erlingsson, John C Mitchell, Mark S Miller, and Jasvir Nagra. Automated analysis of security-critical javascript apis. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2011.

[25] Google Open Source Security Team. Supply-chain levels for software artifacts (slsa) framework specification. In *Linux Foundation OpenSSF Technical Report*, 2023.

[26] Nirmata Engineering Team. Kyverno: Kubernetes native policy management engine. In *Cloud Native Computing Foundation (CNCF)*, 2022.

[27] Santiago Torres-Arias, Anish Kathpal, Hiten Gupta, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410, 2019.

[28] John Yang, Carlos E Jimenez, Alexander Wettig, Tatiana Likhomanenko, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023.

A Open Science Appendix

The reproducibility materials consist of the complete EviAssure implementation (including cryptographic attestation, privacy blinding, and policy verification modules), the 1,050-profile agent execution trace corpus, the 12-vector tamper evaluation harness, production GitHub Action workflows, benchmark scripts, and the paper figure generator.

A.1 Anonymous Review Artifact & Verification

To comply with USENIX Security '27 Open Science guidelines while preserving double-blind review, an anonymous review package `eviassure_usenix27_artifact.zip` (size: 153.9 KB) has been prepared using `scripts/prepare_anonymous_artifact.py`. The archive carries SHA-256 digest:

`c980d860059b0995...`

A.2 Reproduction Workflow

To execute full reproduction of paper figures, benchmark results, and unit tests:

1. **Environment Setup**: Install dependencies via `pip install -e .` (Python $\geq$ 3.10).
2. **Unit & Integration Test Suite**: Run `pytest tests/ -v` to execute all 25 unit, integration, browser attestation, trusted-key-registry, and tamper resilience tests.
3. **Attestation Scaling & Throughput Microbenchmarks**: Run `python3 scripts/run_release_benchmark.py` to generate `results/benchmark_summary.json`.
4. **Adversarial Tamper Evaluation**: Run `python3 scripts/run_comparative_eval.py` to execute the 12-vector attack harness and produce `results/comparative_evaluation.json`.
5. **Figure Generation & Manuscript PDF Rebuilding**: Run `python3 scripts/generate_paper_pdf.py` to regenerate paper plots and recompile `docs/usenix_paper_manuscript.pdf`.

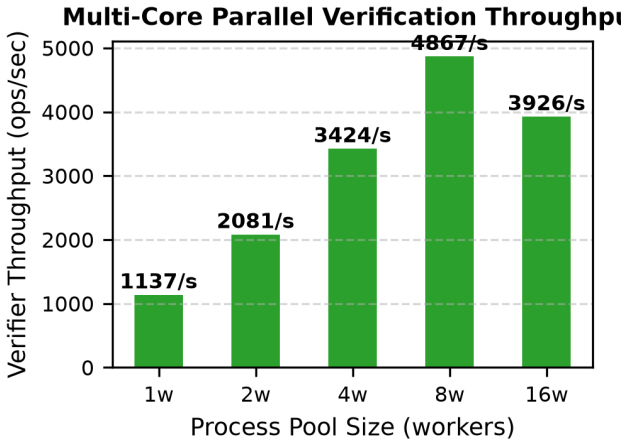

Figure 2: Multi-core parallel policy gate verification throughput across 1, 2, 4, 8, and 16 worker cores.

Table 1: Adversarial Release Tamper Vector Benchmark Results.

ID	Attack Vector	Cat.	Result	Status
V1	Unsigned Payload	Auth	Blocked	PASS
V2	Tampered Trace Digest	Integrity	Blocked	PASS
V3	Forged Signature	Auth	Blocked	PASS
V4	Replayed Nonce	Replay	Blocked	PASS
V5	Expired Timestamp	Fresh	Blocked	PASS
V6	Unresolved Drift	Policy	Blocked	PASS
V7	Sub-threshold Pass Rate	Quality	Blocked	PASS
V8	Forged Merkle Root	Integrity	Blocked	PASS
V9	Revoked Key ID	Key Gov	Blocked	PASS
V10	JSON Malleability	Serial	Blocked	PASS
V11	Future Clock Skew	Fresh	Blocked	PASS
V12	Truncated Merkle Path	Integrity	Blocked	PASS

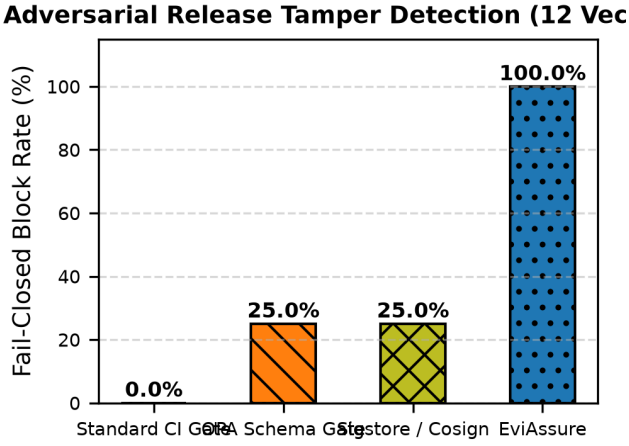

Figure 3: Comparative release tamper detection block rates across 12 attack vectors.

Table 2: Comparative analysis of EviAssure against existing software supply chain and policy gate frameworks.

System	Trace Attest.	Asym. Signing	SLSA v1.0	Fail-Closed	Replay Prot.	Agent-Spec.
SLSA Framework [25]	Baseline	Yes	Yes	Manual	Partial	No
in-toto [27]	Step-level	Yes	Partial	Manual	No	No
Sigstore / Cosign [17]	Artifact-level	Yes	Envelope	No	Partial	No
Open Policy Agent (OPA) [23]	No	Optional	No	Optional	Manual	No
Kyverno Policy Engine [26]	No	Cosign	No	Enforced	No	No
Macaroons Authorization [3]	Dynamic	HMAC	No	Enforced	Token	No
SPIFFE / SPIRE Workload ID [9]	Identity	X.509	No	Enforced	Short TTL	No
Static Security Analysis [24]	Model-based	No	No	Static	No	No
EviAssure	**Merkle (N=1M)**	**Ed25519**	**Full Envelope**	**Fail-Closed**	**Nonce+Time**	**Yes**

Table 3: Adversary capability classes mapped to evaluation vectors and enforcement.

Class	Vectors	Enforcement
$\mathcal{A}_1$ Trace tampering	V2, V8, V12	Merkle root re-derivation over all leaves
$\mathcal{A}_2$ Signature forgery	V1, V3	Ed25519 over canonical payload; trusted-key registry
$\mathcal{A}_3$ Replay / skew	V4, V5, V11	Nonce cache (validity window) + freshness bounds
$\mathcal{A}_4$ Key governance	V9	CRL membership; KMS ARN boundary pattern
$\mathcal{A}_5$ Malleability	V10	Canonical JSON serialization in signed payload
Policy compliance	V6, V7	Drift and pass-rate thresholds